

The Vibe Coding Playbook

PRINTMAXX | printmaxxer.gumroad.com

This product is for personal use only. Do not redistribute.

The Vibe Coding Playbook: Build Production Apps with AI (Zero Coding Required)

Price: \$47

What this is

A complete system for building real, shippable apps by describing what you want in plain English. AI writes the code. You direct the project.

This isn't a "learn to code" course. You don't need to know what a function is, what an API means, or how React works. You describe the app. The AI builds it. You launch it.

5 complete app walkthroughs with exact prompts. Tool setup guide. App Store submission process. Monetization strategies. Common pitfalls and how to fix them without touching code.

Karpathy called it "vibe coding." It crossed the threshold in 2025. By 2026, non-coders are shipping apps that make real money. This playbook shows you exactly how.

Who this is NOT for

- Professional developers (you already know this)
- People who want to "learn programming" (this bypasses programming)
- Anyone expecting passive income from day 1 (apps need marketing too)

Who this IS for

- Non-technical founders with app ideas they can't build
 - Solopreneurs who want to add apps to their revenue mix
 - Content creators who want to turn their audience into app users
 - Anyone who's paid \$5K-\$50K for app development and got burned
-

Part 1: What vibe coding actually is (and why it works now)

The shift

Traditional app development (2010-2024):

1. Learn Swift/Kotlin/React Native (3-6 months)
2. Or hire developers (\$15,000-\$100,000)
3. Manage sprints, tickets, code reviews
4. Debug for weeks
5. Ship after 3-12 months

Vibe coding (2025+):

1. Describe your app in English
2. AI writes the code
3. Test in simulator
4. Fix issues by describing the problem in English
5. Ship in hours to days

Same output. 10-100x faster. Zero technical knowledge required.

Why it works now (and didn't work 2 years ago)

Three things changed:

1. AI code quality crossed the "production" line.

In 2023, AI code was a novelty. Good for snippets, bad for full apps. By late 2025, Claude and GPT-4 write production-quality code that handles edge cases, error states, and real-world usage patterns. The gap between AI code and human developer code is now negligible for standard apps.

2. AI coding tools got real UIs.

You don't use raw ChatGPT to vibe code. You use purpose-built tools like Claude Code (terminal-based, handles full project structure) and Cursor (VS Code fork with AI built in). These tools understand your project context, not just individual prompts.

3. The app stores don't care who wrote the code.

Apple and Google don't reject apps because AI wrote them. They reject apps for quality issues, policy violations, and crashes. If the app works, passes review, and doesn't violate policies, it ships. How it was built is irrelevant.

The Karpathy moment

Andrej Karpathy (co-founder of OpenAI, former Tesla AI director) publicly described "vibe coding" in early 2025:

"There's a new kind of coding I call vibe coding, where you fully give in to the vibes, embrace exponentials, and forget that the code even exists."

When one of the world's top AI researchers says non-coders can build real software, the threshold has been crossed. This isn't a prediction. It's a report.

Part 2: Setting up your vibe coding environment (30 minutes)

You need 3 things. Total cost: \$20-40/month.

Tool 1: Claude Code (primary builder)

Claude Code is a terminal-based AI coding tool. You describe what you want. It builds it.

Setup:

1. Install Node.js from nodejs.org (click the big green button, install, done)
2. Open Terminal (Mac: Cmd+Space, type "Terminal", Enter)
3. Run: `npm install -g @anthropic-ai/claude-code`
4. Run: `claude` to start
5. Sign up for Claude Max (\$100/month -- this gives unlimited Opus usage)

Why Claude Code over ChatGPT:

- Understands your entire project (not just one file)
- Creates, edits, and deletes files automatically
- Runs your app and sees errors itself
- Fixes bugs in context (knows what came before)

Cost: \$100/month for Claude Max (unlimited). Or \$20/month for Claude Pro with usage limits.

Tool 2: Xcode (for iOS apps)

If building iOS apps:

1. Open Mac App Store
2. Search "Xcode"
3. Install (it's free, but 12GB download)
4. Open it once and accept the license agreement

That's it. Claude Code does the actual Xcode project setup.

Tool 3: Cursor (alternative/supplement)

Cursor is a code editor with AI built in. Good for when you want to SEE the code and make targeted changes.

1. Download from cursor.com
2. Install
3. Sign in (free tier works, Pro is \$20/month)
4. Open your project folder

When to use which:

- Claude Code: Building from scratch, major features, complex changes
- Cursor: Small tweaks, UI adjustments, quick fixes

Optional: iOS Simulator

After installing Xcode, you can test apps without a physical phone:

1. Open Xcode
 2. Go to Xcode > Open Developer Tool > Simulator
 3. Pick an iPhone model
 4. Your app runs on a virtual iPhone on your screen
-

Part 3: The vibe coding process (the actual method)

The 5-step loop

Every feature, every bug fix, every app follows this same loop:

```
DESCRIBE -> REVIEW -> TEST -> FIX -> SHIP
```

Step 1: DESCRIBE -- Tell the AI what you want in plain English

Step 2: REVIEW -- Look at what it built (does it make sense?)

Step 3: TEST -- Run it in the simulator or browser

Step 4: FIX -- Describe any problems, let AI fix them

Step 5: SHIP -- When it works, move to the next feature

You repeat this loop for every feature until the app is complete.

The critical skill: Writing good descriptions

This is the only skill you need to develop. The better your descriptions, the better the output.

Bad description:

```
Make a fitness app
```

Good description:

```
Build an iOS app called WalkToUnlock.
```

```
Core feature: The app locks the user's phone until they hit their daily step goal. It connects to Apple HealthKit for step counting.
```

```
Screen 1 - Home: Shows current step count, daily goal, progress bar, and a large "Lock Phone" button. When locked, the screen shows a motivational quote and the remaining steps needed.
```

```
Screen 2 - Settings: Let the user set their daily goal (1000, 3000, 5000, or 10000 steps). Toggle for morning reminder notifications. Dark mode toggle.
```

```
Screen 3 - Stats: 7-day step history as a bar chart. Current streak (consecutive days meeting goal). Total steps this month.
```

```
Design: Clean, minimal. White background. Accent color: #22C55E (green).
```

SF Pro font. Large numbers, small labels.

Monetization: Free version has 3 locks per day. Premium (\$4.99/month) gives unlimited locks, all themes, and detailed analytics.

The difference: Specificity. Screen-by-screen. Feature-by-feature. Colors, fonts, numbers. The more specific you are, the less back-and-forth with the AI.

The prompt structure formula

Use this template for every new app or major feature:

```
[WHAT]: Build [type of thing] called [name]

[WHO]: For [target user] who wants [core benefit]

[SCREENS]:
- Screen 1: [name] - [what it shows and does]
- Screen 2: [name] - [what it shows and does]
- Screen 3: [name] - [what it shows and does]

[FEATURES]:
- [Feature 1]: [specific behavior]
- [Feature 2]: [specific behavior]
- [Feature 3]: [specific behavior]

[DESIGN]: [style], [colors], [font], [layout approach]

[DATA]: [what gets saved, where, how]

[MONETIZATION]: [free tier limits, paid tier features, price]
```

Fixing bugs without code knowledge

When something doesn't work, describe the problem:

Bad bug report:

```
It's broken
```

Good bug report:

```
When I tap the "Lock Phone" button, the app crashes instead of
showing the lock screen. This happens every time, on both the
simulator and my real phone. The step counter on the home screen
works fine - it's only the lock button that crashes.
```

The AI will find the bug and fix it. You don't need to understand the fix. You just need to accurately describe what went wrong.

When the AI gets stuck

Sometimes the AI writes code that doesn't work and can't figure out why. This happens maybe 10-15% of the time for complex features.

Strategy 1: Start over with that feature

Delete the lock screen feature entirely and rebuild it from scratch.
 This time, start with the simplest possible version: just a full-screen view that says "Phone Locked" with a close button. Get that working first.
 Then add the step counter display. Then add the motivational quotes.
 Build up one layer at a time.

Strategy 2: Simplify

The current approach is too complex. Let's simplify.
 Instead of [complex version], just do [simple version].
 We can add the complexity later once the basics work.

Strategy 3: Research mode

Before writing any code, explain 3 different approaches to implementing [feature]. For each approach, list the pros, cons, and potential issues.
 I'll pick the approach, then you implement it.

This forces the AI to think before coding, which often produces better results.

Part 4: Full app walkthrough -- PrayerLock (faith niche)

This walkthrough shows the exact prompts used to build a complete app from zero to App Store ready. Follow along to build your own version, or adapt the process for your own app idea.

Session 1: Project setup and home screen (45 minutes)

Prompt 1 -- Project creation:

Create a new iOS app project called PrayerLock using SwiftUI.

The app helps Muslims and Christians maintain consistent prayer habits by locking their phone during prayer time.

Set up the project with:

- SwiftUI navigation using NavigationStack
- Tab bar with 3 tabs: Home, Timer, Settings
- App icon placeholder
- Color scheme: Deep blue (#1E3A5F) primary, gold (#D4AF37) accent
- SF Pro font throughout

Start with the Home tab showing:

- "PrayerLock" title
- Current prayer streak (days in a row)
- Large "Start Prayer" button
- Today's prayer count (e.g., "3 of 5 prayers today")

What happens: Claude Code creates the entire Xcode project structure, writes SwiftUI views for all 3 tabs, sets up navigation, and applies the color scheme. You'll see it creating files in real-time.

Prompt 2 -- Test and refine:

Run this in the iOS Simulator. Show me how to preview it.

Claude Code will tell you how to open it in Xcode and run the simulator. You'll see your app on a virtual iPhone screen.

Prompt 3 -- Visual adjustments:

The home screen looks too cramped. Add more spacing between elements.
Make the "Start Prayer" button bigger - at least 60pt height with rounded corners. The streak number should be large (48pt font) and the gold accent color. Add a subtle gradient background from dark blue to slightly lighter blue.

Session 2: Prayer timer feature (60 minutes)

Prompt 4 -- Timer screen:

Build the Timer tab with this exact flow:

1. User sees duration options: 5, 10, 15, 30 minutes (as large tappable cards, not small buttons)
2. User taps a duration -> countdown screen appears:
 - Large countdown timer (centered, 72pt font)
 - Circular progress ring around the timer
 - "End Early" button at bottom (small, de-emphasized)
 - Random inspirational quote from a pre-loaded list of 20 quotes
 - Quote changes every 60 seconds
3. When timer completes:
 - Gentle chime sound
 - "Prayer Complete" celebration screen
 - Add 1 to today's prayer count
 - Update streak if this is first prayer today
 - "Back to Home" button
4. Data persistence: Save prayer history using SwiftData.
Track: date, duration, time of day.

Prompt 5 -- Quote library:

Add 20 inspirational quotes to the prayer timer. Mix of:

- 7 Islamic quotes (Quran verses with citation)
- 7 Christian quotes (Bible verses with citation)
- 6 universal spiritual quotes about peace and mindfulness

Store these in a simple array in the app. The app should randomly select one to display and rotate every 60 seconds during prayer.

Prompt 6 -- Sound:

Add a gentle chime sound when the prayer timer completes.
 Use the system sound API (no custom audio files needed).
 Also add a brief haptic feedback (medium impact) when the timer starts and when it ends.

Session 3: Settings and premium features (45 minutes)

Prompt 7 -- Settings screen:

Build the Settings tab with these options:

1. Prayer reminder notifications
 - Toggle on/off
 - If on, show time picker for each of 5 daily reminders
 - Default times: 5:30 AM, 12:30 PM, 3:30 PM, 6:30 PM, 8:30 PM
2. Theme selection
 - Islamic theme (blue + gold, crescent moon icon)
 - Christian theme (purple + white, cross icon)
 - Universal theme (teal + white, lotus icon)
 - Theme changes the accent color throughout the app
3. Premium section
 - Show current plan (Free or Premium)
 - If free: show "Upgrade to Premium" button
 - Premium features list: unlimited prayers, all themes, detailed analytics, no ads
 - Price: \$4.99/month or \$29.99/year
4. About section
 - App version
 - Privacy policy link
 - Terms of service link
 - Contact email

Prompt 8 -- In-app purchase:

Set up RevenueCat for in-app subscriptions.

Two products:

- Monthly: \$4.99/month (product ID: premium_monthly)
- Annual: \$29.99/year (product ID: premium_annual)

Free tier limits:

- 3 prayer sessions per day
- 1 theme (based on initial selection)
- Basic stats only

Premium tier:

- Unlimited prayer sessions
- All 3 themes
- Detailed analytics (weekly/monthly trends)
- No ads

Add a paywall screen that shows when free user hits the 3-prayer daily limit. Show both pricing options with the annual option highlighted as "Best Value - Save 50%."

Use RevenueCat SDK for purchase handling. I'll add my RevenueCat API key later.

Session 4: Polish and App Store prep (60 minutes)

Prompt 9 -- Analytics screen:

Replace the basic stats with a proper analytics view (premium only, free users see a blurred preview with "Upgrade" overlay):

- This week: Bar chart showing prayers per day (Mon-Sun)
- This month: Calendar view with color-coded days (green = hit goal, yellow = partial, grey = missed)
- Streak counter with flame emoji animation
- Total prayers all-time
- Average prayers per day
- Most consistent time of day

Use Swift Charts for the bar chart. Make it look clean and modern.

Prompt 10 -- App Store assets:

Help me prepare for App Store submission:

1. Generate an app description (max 4000 chars):
 - First line is the most important (visible without expanding)
 - Include keywords: prayer, faith, accountability, focus, meditation, spiritual, worship
 - List key features as bullet points
 - End with the free/premium breakdown
2. Generate 3 keyword sets to test:
 - Set A: Broad (prayer app, faith, spiritual)
 - Set B: Niche (Islamic prayer, Christian devotional, prayer timer)
 - Set C: Problem-based (phone addiction, focus, discipline)
3. Generate the "What's New" text for version 1.0
4. Tell me exactly what screenshots I need and what each should show

Prompt 11 -- Final polish:

Review the entire app for these issues:

1. Any screens that crash or show errors
2. Dark mode compatibility (does everything look good in dark mode?)
3. Accessibility (is VoiceOver supported? Are fonts dynamic type ready?)
4. Memory leaks or performance issues
5. Missing error states (what happens with no internet? low storage?)

Fix anything you find.

What you have after 4 sessions (~3.5 hours)

A complete iOS app with:

- Home screen with prayer tracking
- Prayer timer with quotes and sounds
- Settings with theme selection and notifications
- In-app subscription (RevenueCat)
- Analytics dashboard
- App Store optimized description and keywords

Total lines of code written by you: zero.

Total cost: \$0 (just your Claude subscription you already have).

Part 5: Full app walkthrough -- WalkToUnlock (fitness niche)

Same process, different app. Showing you it's repeatable.

The prompts (condensed -- same structure as PrayerLock)

Prompt 1 -- Setup:

```
Create a new iOS app called WalkToUnlock using SwiftUI.
```

```
The app motivates users to walk more by locking certain phone features until they hit their daily step goal.
```

```
Core mechanic: User sets a step goal. App connects to Apple HealthKit to track steps. When the user hasn't met their goal, they see a lock screen overlay encouraging them to walk. Once the goal is hit, a celebration screen appears and the lock is removed.
```

```
Tab bar: Home, Progress, Settings
```

```
Color: #22C55E (green) primary, white background
```

```
Modern, clean design with large typography
```

Prompt 2 -- HealthKit integration:

```
Connect to Apple HealthKit for step counting.
```

```
Request step count permission on first launch.
```

```
Show a clear explanation: "WalkToUnlock reads your step count to track your daily walking goal. We never access any other health data."
```

```
Home screen should display:
```

- Current steps today (large number, updates live)
- Daily goal (smaller, below current steps)
- Circular progress ring (fills as you approach goal)
- "Lock Apps" toggle button

When locked:

- Show a motivational overlay on the home screen
- Display: steps remaining, motivational quote, ETA to goal (based on current pace)
- The overlay disappears automatically when the goal is hit

Prompt 3 -- Progress tab:

Build the Progress tab:

- 7-day bar chart (steps per day, goal line shown as dashed horizontal)
- Current streak (consecutive days meeting goal)
- Best streak ever
- Total steps this week vs. last week (with % change)
- Monthly calendar view with green/red dots (met goal or didn't)

Use Swift Charts. Make the weekly bar chart the hero element at the top.

Prompt 4 -- Gamification:

Add achievement badges to the Progress tab:

- "First Step" - Complete your first day's goal
- "Week Warrior" - 7-day streak
- "Marathon Month" - 30-day streak
- "10K Club" - Hit 10,000 steps in a day
- "Early Bird" - Meet your goal before noon
- "Night Owl" - Meet your goal after 8 PM
- "Overachiever" - Double your daily goal

Show badges as a horizontal scrolling grid.

Locked badges are greyed out with a lock icon.

Unlocked badges have a subtle glow animation.

Tap a badge to see how you earned it and when.

Prompt 5 -- Settings and monetization:

Settings tab:

1. Daily step goal: Slider from 1,000 to 20,000 (default 5,000)
2. Reminder notifications: Toggle + time picker
3. Lock intensity:
 - Gentle: Motivational overlay (dismissable)
 - Medium: Overlay stays until 50% of goal met
 - Strict: Overlay stays until 100% of goal met
4. Premium (\$4.99/month or \$29.99/year):
 - All achievement badges
 - Detailed analytics (hourly step breakdown)
 - Custom themes
 - Widget for home screen
5. About section

Time to build: ~3 hours following the same 4-session approach.

Part 6: Three more app concepts (build your own)

These are templates. Fill in the specifics for your niche.

StudyLock (student niche)

iOS app called StudyLock.

Pomodoro-based study timer that locks distracting apps during study sessions.

Home: Subject picker + "Start Study Session" button.

Timer: 25-minute countdown with 5-minute breaks. Shows current subject, session count today, and a "focus score" based on how many times you tried to leave the app.

Stats: Daily/weekly study hours by subject. Exam countdown timers.

Settings: Session duration (15/25/45/60 min), break duration (5/10/15 min), which apps to block during sessions.

Monetization: Free = 3 sessions/day. Premium = unlimited + subject analytics + study group features.

WorkLock (productivity niche)

iOS app called WorkLock.

Schedule-based phone lock for remote workers who need deep focus.

Home: Today's schedule showing locked/unlocked time blocks.

Calendar integration - auto-lock during meetings.

Focus: During locked time, only shows whitelisted apps (Slack, email, calendar). Focus score visible.

Report: Weekly productivity report - hours focused, interruption attempts, most productive day/time.

Settings: Work schedule (start/end time, weekdays only toggle), whitelisted apps, emergency contacts (always accessible).

Monetization: Free = 2 focus blocks/day. Premium = unlimited blocks + calendar sync + team features.

SleepLock (wellness niche)

iOS app called SleepLock.

Bedtime routine builder that locks the phone before sleep and requires morning tasks to unlock.

Home: Tonight's bedtime routine (customizable steps).

Bedtime: 1 hour before sleep target, phone gradually locks.

15-min warning, 30-min dimming, full lock at bedtime.

Morning: Unlock requires completing 3 of 5 tasks: drink water, stretch for 2 min, get sunlight, journal entry, 5 deep breaths.
Stats: Sleep consistency score, morning routine completion rate, average bedtime over past 30 days.

Monetization: Free = basic routine. Premium = custom routines, sleep sounds library, partner sync, advanced analytics.

Part 7: App Store submission guide

Step 1: Apple Developer account (\$99/year)

1. Go to developer.apple.com
2. Enroll in Apple Developer Program
3. Pay \$99/year
4. Wait 24-48 hours for approval

Step 2: Prepare your app

Ask Claude Code:

```
Prepare this app for App Store submission. Run through this checklist:  
1. Remove all debug code and console logs  
2. Set the app version to 1.0.0 and build number to 1  
3. Add required privacy descriptions for any permissions used  
4. Create the app icon in all required sizes  
5. Verify the app works in both light and dark mode  
6. Test on iPhone 15 Pro and iPhone SE screen sizes  
7. Make sure all error states are handled
```

Step 3: Screenshots (required)

You need screenshots for:

- 6.7" (iPhone 15 Pro Max) -- 1290 x 2796 pixels
- 6.5" (iPhone 14 Plus) -- 1284 x 2778 pixels
- 5.5" (iPhone 8 Plus) -- 1242 x 2208 pixels

Take them in the Simulator. Add text overlays in Canva to highlight features.

Minimum 3 screenshots, maximum 10. Use 5-6 for best results:

1. Main feature / hero screen
2. Key differentiator
3. Stats/progress view
4. Settings/customization
5. Premium features preview

Step 4: Submit through App Store Connect

1. Go to appstoreconnect.apple.com
2. Create new app listing
3. Fill in: name, subtitle, description, keywords, category
4. Upload screenshots
5. Upload build (Claude Code can walk you through this with Xcode)
6. Set pricing (free with IAP)
7. Submit for review

Step 5: Handle rejection (it happens)

Common rejection reasons and fixes:

- Guideline 2.1 (Performance): App crashes. Fix bugs and resubmit.
- Guideline 2.3 (Accurate metadata): Screenshots don't match app. Retake screenshots.
- Guideline 3.1.1 (In-App Purchase): IAP not configured properly. Check RevenueCat setup.
- Guideline 4.0 (Design): UI doesn't meet quality bar. Polish the design.
- Guideline 5.1.1 (Privacy): Missing privacy descriptions. Add all required permission explanations.

Average: 1-2 rejections before approval. Each revision takes 24-48 hours for re-review.

Part 8: Monetization strategies

Strategy 1: Freemium (recommended for most apps)

Free tier: Core features with daily limits (3 uses/day)

Premium tier: Unlimited usage + analytics + themes

Pricing:

- Monthly: \$4.99
- Annual: \$29.99 (50% discount, highlighted as "Best Value")
- Lifetime: \$49.99 (optional, good for early traction)

Conversion benchmarks:

- 2-3% free-to-paid is average
- 5-7% is good
- 10%+ is excellent (usually means free tier is too restrictive)

Strategy 2: Hard paywall

No free tier. 3-day free trial, then paywall.

Works best for:

- Apps with strong word-of-mouth
- Apps solving an urgent problem
- Apps with unique value (no free alternatives)

Strategy 3: One-time purchase

No subscription. Pay once, own forever.

- Price: \$4.99-\$14.99
- Higher upfront revenue, lower LTV
- Good for simple utility apps
- Users prefer this (subscription fatigue is real)

Revenue projections (conservative)

Downloads/month	Convert rate	Price	Monthly revenue
1,000	3%	\$4.99/mo	\$150
5,000	3%	\$4.99/mo	\$750
10,000	3%	\$4.99/mo	\$1,500
10,000	5%	\$4.99/mo	\$2,500
25,000	5%	\$4.99/mo	\$6,250

Key growth levers:

- ASO (App Store Optimization) drives organic downloads
- Social content drives awareness
- Each additional app cross-promotes the others

With 5 apps each getting 5,000 downloads/month at 3% conversion: \$3,750/month MRR. All from apps you didn't write a single line of code for.

Part 9: Common pitfalls and fixes

Pitfall 1: "The AI keeps making the same mistake"

Fix: Start a new Claude Code session. Copy the current state of your project but describe the problem fresh. Sometimes the AI gets stuck in a loop with its own context. Fresh context = fresh approach.

Pitfall 2: "The app looks ugly"

Fix: Be more specific about design in your prompts. Instead of "make it look good," say:

Use 16px padding on all sides. 8px spacing between elements.
 Font sizes: titles 24pt bold, body 16pt regular, labels 12pt.
 Rounded corners: 12px on cards, 8px on buttons.
 Shadow: 0 2px 8px rgba(0,0,0,0.1) on cards.

Pitfall 3: "I have too many ideas and can't finish one"

Fix: Commit to one app. Finish it. Ship it. Then start the next. The first app teaches you the process. The second takes half the time. By the fifth, you're doing it in your sleep.

Pitfall 4: "The app works in simulator but crashes on real device"

Fix: Ask the AI:

```
The app works in the simulator but crashes on my physical iPhone.  
Review the code for common simulator-vs-device differences:  
- Are we using any simulator-only APIs?  
- Are permissions being requested properly?  
- Is the app handling low memory situations?  
- Are file paths hardcoded to simulator locations?  
Fix any issues you find.
```

Pitfall 5: "I want to add more features but it's getting complicated"

Fix: Ship the MVP first. Get real users. Let them tell you what features they want. The features you think are important and the features users actually want are usually different. Ship simple, iterate based on feedback.

Pitfall 6: "Apple rejected my app"

Fix: Read the rejection reason carefully. It's specific. Ask the AI:

```
Apple rejected my app with this message:  
"[PASTE REJECTION MESSAGE]"  
  
What changes do I need to make to pass review?  
Make all required changes.
```

90% of rejections are fixable in under an hour.

Part 10: Scaling beyond one app

The portfolio approach

One app is a side project. Five apps is a business.

Each app:

- Serves a different niche (faith, fitness, productivity, wellness, education)
- Cross-promotes the others ("From the makers of PrayerLock")
- Shares the same monetization infrastructure (RevenueCat)
- Generates independent revenue streams

If one app underperforms, the others carry it. Diversification in app form.

The content multiplier

Every app you build generates 15+ content pieces:

1. Build-in-public Twitter thread
2. YouTube walkthrough video
3. "How I built this" blog post
4. TikTok/Reels clips from the build process
5. ProductHunt launch
6. Reddit post (r/SideProject, r/IndieHackers)
7. LinkedIn case study
8. Newsletter feature
9. Podcast talking point
10. Medium technical article

5 apps x 15 pieces = 75 content assets. The content drives downloads, which drives revenue, which funds more apps.

The playbook meta-play

You're reading a playbook about building apps. Once you've built your own apps, you can sell your own playbook. Document your process. Share your results. Teach others.

The playbook itself becomes a revenue stream:

- \$47 per sale
- 100 sales/month = \$4,700
- Plus the app revenue
- Plus consulting inquiries from people who want you to build their apps

This is the real flywheel: Build apps -> Document process -> Sell playbook -> Get consulting leads -> Build more apps.

Part 11: Prompt library (copy-paste ready)

New project setup

```
Create a new [iOS/web] app called [NAME] using [SwiftUI/React/Next.js].
```

```
Purpose: [ONE SENTENCE]
```

```
Target user: [WHO]
```

```
Core feature: [THE ONE THING IT DOES]
```

```
Tabs/pages:
```

```
- [Tab 1]: [description]
```

```
- [Tab 2]: [description]
```

- [Tab 3]: [description]

Design: [style], [primary color hex], [accent color hex], [font]

Add a feature

Add [FEATURE NAME] to [SCREEN/TAB].

Behavior:

1. [What triggers it]
2. [What the user sees]
3. [What data is saved]
4. [What happens when it's done]

Design: [size, position, color, animation]

Fix a bug

Bug: [EXACT DESCRIPTION OF WHAT GOES WRONG]

Expected: [WHAT SHOULD HAPPEN]

When: [WHEN DOES IT HAPPEN - every time? sometimes? after X?]

Where: [WHICH SCREEN/FEATURE]

Find and fix this bug.

Improve design

The [SCREEN] looks [PROBLEM: cluttered/boring/cramped/inconsistent].

Make these changes:

- Spacing: [specific pixel values]
- Font sizes: [specific pt values]
- Colors: [specific hex values]
- Layout: [specific arrangement]
- Animation: [specific transitions]

App Store prep

Prepare for App Store submission:

1. Remove all debug/test code
2. Set version to [X.Y.Z]
3. Add privacy descriptions for: [permissions used]
4. Generate app icon in all required sizes
5. Write App Store description (max 4000 chars)
6. Suggest 100 characters of keywords
7. Test on [device sizes]

Monetization setup

Add in-app subscriptions using RevenueCat:

Products:

- Monthly: \$[X.XX]/month (ID: [product_id])
- Annual: \$[X.XX]/year (ID: [product_id])

Free tier limits: [WHAT'S RESTRICTED]

Premium features: [WHAT'S UNLOCKED]

Paywall screen: Show when free user hits [LIMIT].

Display both options, highlight annual as "Best Value."

Part 12: Web apps (not just iOS)

Everything above focuses on iOS apps. But vibe coding works for web apps too. Same process, different output.

Why web apps

- No Apple Developer fee (\$99/year saved)
- No App Store review process (ship instantly)
- Works on every device (phone, tablet, desktop)
- Easier to update (push code, done -- no review wait)
- Lower barrier: users don't need to install anything

Web app tech stack

Tell Claude Code to use:

- Next.js for the framework (React-based, handles routing, SEO, hosting)
- Tailwind CSS for styling (utility-first, fast to build with)
- Vercel for hosting (free tier handles most apps)
- Supabase for database + auth (free tier: 50K monthly active users)

Cost: \$0 to start. Scales to \$20-50/month when you hit real traffic.

Web app project setup prompt

Create a Next.js web app called [NAME] using the App Router and Tailwind CSS.

Purpose: [ONE SENTENCE]

Target user: [WHO]

Pages:

- / (landing page): Hero section with headline, subheadline, CTA button. Features section with 3 cards. Social proof section. Footer with links.
- /app (main app): [DESCRIBE THE CORE FUNCTIONALITY]
- /pricing: Free vs Premium comparison table. Stripe checkout button.

- /login: Email + password auth using Supabase.
- /dashboard: User's data, settings, account management.

Design: Modern, clean. [PRIMARY COLOR]. [ACCENT COLOR].
Inter font. Responsive (mobile-first).

Set up Supabase for:

- User authentication (email/password)
- Database tables for [WHAT DATA YOU NEED TO STORE]
- Row-level security so users can only see their own data

Example: SaaS calculator tool

Create a Next.js web app called "SaaS Metrics Calculator."

Landing page: "Know your real SaaS numbers in 60 seconds."
Headline, 3 feature cards (MRR calculator, churn analysis, LTV predictor), testimonial section, email capture form.

App page (no login required):

- Input fields: monthly revenue, total customers, new customers this month, churned customers this month, average revenue per customer
- Auto-calculates and displays: MRR, ARR, churn rate, LTV, CAC ratio, months to profitability
- Results update in real-time as inputs change
- "Save report" button (requires free account)
- "Generate PDF report" button (premium, \$9.99 one-time)

Design: Dark theme. #0F172A background. #3B82F6 accent.
Cards with subtle border. Large numbers for results.

This type of tool app is perfect for lead generation. Free calculator gets traffic, email capture builds your list, premium PDF report generates revenue.

Example: Directory/marketplace

Create a Next.js web app called "[NICHE] Directory."

Landing page: Browse [CATEGORY] tools/services with filters.
Search bar at top, category filters on sidebar, card grid showing listings.

Each listing card shows: name, short description, category, price range, rating, link.

Listing detail page: Full description, screenshots, pricing breakdown, user reviews, "Visit Website" button with affiliate tracking.

Admin page (password protected): Add/edit/remove listings.
Bulk import from CSV.

Data: Store in Supabase. Fields: name, description, category, price_range, url, affiliate_url, rating, image_url.

Monetization:

- Featured listings: \$49/month (highlighted, shown first)
- Affiliate commission on clicks that convert
- Sponsored posts: \$99/post in the newsletter

Directory sites are underrated. Low effort to maintain, high SEO value, passive revenue from affiliates and featured listings.

Deploying web apps (5 minutes)

Deploy this app to Vercel:

1. Initialize a git repository
2. Create a Vercel account at vercel.com
3. Connect the GitHub repo
4. Set environment variables for Supabase keys
5. Deploy

Claude Code walks you through the entire process. Your app goes live with a .vercel.app URL. Add a custom domain for \$10-15/year.

Full web app walkthrough: Habit Tracker SaaS

This walkthrough builds a complete web app from zero to deployed.

Session 1: Landing page + core app (60 minutes)

Prompt 1 -- Project setup:

Create a Next.js 14 web app called "HabitStack" using the App Router, Tailwind CSS, and TypeScript.

HabitStack helps people build and track daily habits with streak counting and accountability.

Pages to create:

1. / (landing page):
 - Hero: "Build habits that stick. Track your streaks. Stay accountable." Big heading, subtext, "Start Free" CTA button.
 - Feature cards (3): Streak tracking, daily reminders, progress analytics
 - Social proof: "Join 2,000+ habit builders" with 3 fake testimonial cards
 - Footer: links, copyright
2. /app (main dashboard, requires auth):
 - "Today's Habits" list showing each habit with a checkbox
 - Completed count: "3 of 5 done today"
 - "Add Habit" button that opens a modal
 - Each habit shows: name, current streak, emoji icon

Design: White background. #8B5CF6 (purple) primary. #F59E0B (amber) accent for streaks. Inter font. Rounded cards with subtle shadows. Mobile-responsive.

Prompt 2 -- Database and auth:

Set up Supabase for HabitStack:

1. Auth: Email/password signup and login.
 - /login page with email + password form
 - /signup page with same form + "confirm password"
 - Redirect to /app after successful auth
 - Show login/signup links in header when logged out
 - Show "Dashboard" link + avatar when logged in
2. Database tables:
 - habits: id, user_id, name, emoji, created_at, is_active
 - completions: id, habit_id, completed_date
 - (One completion record per habit per day)
3. Row-level security: Users only see/edit their own habits.
4. Queries needed:
 - Get all habits for current user
 - Mark habit as complete for today
 - Unmark habit for today
 - Calculate streak for each habit
 - Get completion history for past 30 days

Prompt 3 -- Streak and analytics:

Add a /stats page with these analytics:

1. Overall completion rate (% of habits completed this week)
2. Habit heatmap: 30-day grid for each habit (green = done, grey = missed, like GitHub's contribution graph)
3. Streak leaderboard: Habits ranked by current streak length
4. Best day of the week: Which day you're most consistent
5. Weekly trend: Line chart showing completion rate over past 8 weeks

Use Recharts for the line chart. Build the heatmap with CSS grid. Link from the dashboard with a "View Stats" button.

Prompt 4 -- Monetization:

Add a freemium model:

Free tier:

- Up to 3 habits
- Basic streak tracking
- 7-day history

Premium (\$4.99/month):

- Unlimited habits

- Full analytics and heatmap
- 365-day history
- Email reminders
- Export to CSV

Add a /pricing page with a comparison table.

Add Stripe checkout for premium upgrade.

When free users try to add a 4th habit, show a paywall modal that links to /pricing.

Use Stripe's client-side checkout. I'll add API keys later.

Prompt 5 -- Deploy:

Prepare for deployment:

1. Add environment variables for Supabase URL, Supabase anon key, and Stripe publishable key
2. Create a .env.example file showing what's needed
3. Add a README with setup instructions
4. Deploy to Vercel (walk me through the steps)

What you have: A fully functional SaaS product. Landing page, auth, CRUD operations, analytics, payments. Deployed and live. Built in one afternoon.

Part 13: Advanced vibe coding techniques

Technique 1: Reference apps

Instead of describing from scratch, reference an existing app:

Build an app similar to [EXISTING APP NAME] but for [YOUR NICHE].

Key differences:

- [Feature A] instead of [their feature A]
- [Feature B] added
- [Their feature C] removed
- Design: [YOUR STYLE] instead of [their style]

Core flow is the same: [DESCRIBE THE USER JOURNEY]

This gives the AI a strong mental model to work from. It understands the UX patterns of popular apps and can adapt them.

Technique 2: Screenshot-driven development

If you see a design you like (website, app, screenshot):

I'm attaching a screenshot of a design I want to recreate.

Build this exact layout:

- Same structure and spacing
- Same typography hierarchy
- Same card layout
- Change the colors to [YOUR COLORS]
- Change the content to [YOUR CONTENT]

Claude Code (with vision) and Cursor both accept images. You can point at a screenshot and say "build this." It's remarkably accurate.

Technique 3: Progressive complexity

Don't build the final version first. Build in layers:

Layer 1: Static layout (screens with fake data, no functionality)

Layer 2: Core feature (the one thing the app does)

Layer 3: Data persistence (save/load data between sessions)

Layer 4: Authentication (user accounts, login)

Layer 5: Monetization (paywall, subscriptions)

Layer 6: Polish (animations, edge cases, loading states)

Each layer is one vibe coding session. Test thoroughly between layers. This prevents the "everything breaks" problem that comes from building too many features at once.

Technique 4: Code review without understanding code

Even though you don't write code, you should sometimes ask the AI to explain what it built:

Explain what this codebase does at a high level.

Don't show me code. Just explain:

1. How data flows through the app
2. Where data is stored
3. What happens when the user does [ACTION]
4. What could break and why

This helps you make better decisions about future features. You don't need to understand the code -- you just need to understand the architecture.

Technique 5: Version control for non-coders

Git (version control) is your undo button. Ask Claude Code to set it up:

Initialize git for this project. Commit the current state with message "Initial working version."

From now on, commit after every major feature addition with a descriptive message.

If something breaks badly, you can always say:

Revert to the last working commit. Something broke and I

want to start over from when it was working.

This is your safety net. Use it.

Technique 6: Testing prompts

Before shipping, ask the AI to think like a QA tester:

Act as a QA tester. Try to break this app.

Test these scenarios:

1. What happens if the user enters invalid data?
2. What happens if the network is slow or offline?
3. What happens if the user rapidly taps buttons?
4. What happens with extremely long text inputs?
5. What happens if the user has no data yet (empty states)?
6. What happens on small screens vs large screens?

For each issue you find, fix it.

This catches 80% of bugs before users do.

Part 14: Building a vibe coding business

Service model: Build apps for clients

Once you've built 3-5 apps for yourself, you have a portfolio. Now you can offer app building as a service:

Pricing:

- Simple utility app (1-3 screens): \$500-1,500
- Medium app (4-8 screens, auth, payments): \$2,000-5,000
- Complex app (10+ screens, API integrations, admin panel): \$5,000-15,000

Your cost: 3-20 hours of vibe coding = \$0 in development costs.

Your margin: 90%+.

Where to find clients:

- Twitter (show your portfolio, people will DM)
- Upwork/Fiverr (position as "rapid app development")
- Local businesses (they all want an app but can't afford \$50K)
- Indie hackers who have ideas but no technical co-founder

Teaching model: Sell the skill

The playbook you're reading right now is an example. You can teach vibe coding:

- Course: \$97-497 with video walkthroughs

- Cohort: \$497-997 for live group building sessions
- 1:1 coaching: \$150-300/hour to help someone build their app
- YouTube: Free content that drives course sales

The vibe coding market is growing fast. Non-coders want to build software. You've learned the skill. Now teach it.

Portfolio model: Build and own

The highest-ROI approach. Build apps you own and collect revenue forever:

- 5 apps x \$500/month each = \$2,500/month passive
- 10 apps x \$500/month = \$5,000/month
- Add web apps and you're looking at 20+ products

The compound effect: Each app teaches you something. Each launch builds your audience. Each success funds the next project. After 6 months, you have a portfolio that generates revenue while you sleep.

Quick-start checklist

If you want to build your first app today:

- [] Install Node.js (5 minutes)
- [] Install Claude Code (2 minutes)
- [] Install Xcode (30 minutes, mostly downloading)
- [] Pick your app idea (or use one from Part 6)
- [] Write your first prompt using the template from Part 3
- [] Run the DESCRIBE -> REVIEW -> TEST -> FIX -> SHIP loop
- [] Aim for a working MVP by end of day

Your first app will take longer. Maybe 6-8 hours total across a few sessions. Your second will take 3-4 hours. By your fifth, you'll have the process down to 2-3 hours per app.

The only way to learn vibe coding is to start. Open your terminal. Type claude. Describe what you want to build.

The AI will handle the rest.

Appendix A: 20 app ideas ready to vibe code

Each idea includes the niche, the core mechanic, the monetization model, and estimated build time.

#	App Name	Niche	Core Mechanic	Monetization	Build Time
1	PrayerLock	Faith	Lock phone during prayer	Freemium \$4.99/mo	3-4 hours
2	WalkToUnlock	Fitness	Lock phone until step goal	Freemium \$4.99/mo	3-4 hours
3	StudyLock	Education	Pomodoro with app blocking	Freemium \$4.99/mo	3-4 hours

4	WorkLock	Productivity	Schedule-based focus mode	Freemium \$4.99/mo	3-4 hours
5	SleepLock	Wellness	Bedtime routine + morning	Freemium \$4.99/mo	3-4 hours
6	HabitStack	Wellness	Habit tracker with streaks	SaaS \$4.99/mo	4-5 hours
7	BioMaxx	Fitness	Biohacking protocol tracker	Freemium \$9.99/mo	5-6 hours
8	MealPrep AI	Health	AI meal planning from fridge	SaaS \$10.99/mo	6-8 hours
9	Invoice Ninja Lite	Business	Simple invoicing for freelancers	SaaS \$9.99/mo	5-6 hours
10	Pitch Deck Builder	Business	AI-generated pitch decks	One-time \$19.99	6-8 hours
11	Rent Tracker	Finance	Split rent/utilities with roommates	Freemium	3-4 hours
12	Dog Walk Log	Pets	Track walks, feeding, vet visits	Freemium \$2.99/mo	3-4 hours
13	Gratitude Journal	Wellness	Daily prompts + photo memory	Freemium \$3.99/mo	3-4 hours
14	Recipe Saver	Food	Save recipes from any URL	Freemium \$4.99/mo	4-5 hours
15	Capsule Wardrobe	Fashion	Outfit planner from closet	Freemium \$4.99/mo	5-6 hours
16	Side Hustle Tracker	Business	Track multiple income streams	SaaS \$7.99/mo	4-5 hours
17	Quran Daily	Faith	Daily verse + tafsir + audio	Freemium \$3.99/mo	3-4 hours
18	Gym Log Pro	Fitness	Workout tracker with PR alerts	Freemium \$4.99/mo	4-5 hours
19	Language Flashcards	Education	AI-generated vocab from articles	SaaS \$6.99/mo	5-6 hours
20	Budget Buddy	Finance	Simple envelope budgeting	Freemium \$4.99/mo	4-5 hours

Pick any of these. Use the prompt structure from Part 3. Build it this weekend.

Appendix B: Vibe coding vs. hiring a developer

Factor	Vibe Coding	Hiring a Developer
Cost per app	\$0 (your time only)	\$5,000-\$100,000
Time to MVP	3-8 hours	2-12 weeks
Communication overhead	None (you direct the AI)	Meetings, Slack, specs
Iteration speed	Minutes	Days to weeks
Technical debt	Some (AI can be messy)	Less (if dev is good)
Scalability	Moderate (may need dev later)	High (if built properly)
Learning curve	Low (speak English)	High (manage devs)
Control	Full (you decide everything)	Partial (dev has opinions)
Ongoing cost	\$20-100/month (AI sub)	\$50-200/hour or salary

When to use vibe coding: MVPs, prototypes, utility apps, personal projects, apps under 20 screens, anything where speed matters more than perfection.

When to hire a developer: Complex backends, real-time multiplayer, apps handling sensitive data (healthcare, fintech), apps needing 99.99% uptime, anything with 50+ screens.

Most solopreneurs never need to hire a developer. Vibe coding handles 90% of what you'll want to build.

Appendix C: Resources

AI coding tools:

- Claude Code: Terminal-based, best for full projects
- Cursor: VS Code fork, best for visual editing
- GitHub Copilot: Inline code suggestions (less useful for non-coders)
- Replit: Browser-based, good for web apps, no install needed

Hosting:

- Vercel: Best for Next.js (free tier generous)
- Netlify: Good alternative to Vercel
- Railway: Good for backend APIs
- Supabase: Database + auth (free tier: 50K MAU)

Monetization:

- RevenueCat: In-app subscriptions (iOS/Android)
- Stripe: Web payments
- LemonSqueezy: Digital products + subscriptions

App Store assets:

- Canva: Screenshots with device frames
- Figma: More control over mockups (free tier works)

Learning more:

- Follow @kabortz, @levelsio, @marc_louvion on Twitter for vibe coding examples
- r/SideProject on Reddit for launch feedback
- ProductHunt for launch distribution

Disclaimer: Results not typical. Individual results vary based on effort, market conditions, and other factors.